

Solución Evaluación (1ª instancia)**Duración:** 3 horas**Ejercicio 0: Pregunta individual sobre el laboratorio**

Describe la forma en que implementó el tablero, indicando si utiliza lista de listas o matriz de funtores.

Ejercicio 1 [30 puntos]

Denominaremos descomposición cuadrada de un entero positivo N a un conjunto de números diferentes $[H_1, H_2, \dots, H_k]$, tales que: $H_1^2 + H_2^2 + \dots + H_k^2 = N$

Por ejemplo: $[1, 3]$ es una descomposición cuadrada de 10 ya que $1^2 + 3^2 = 1 + 9 = 10$.

Notar que no siempre existe la descomposición cuadrada de un número, y cuando existe no tiene por qué ser única.

Implementar en Prolog los siguientes predicados que nos permitan encontrar estas descomposiciones:

a) `suma_cuadrados(+L, ?N)` $\leftarrow$ L es una lista de números únicos, N es el resultado de sumar el cuadrado de todos esos números. Ejemplo:

```
?- suma_cuadrados([2,3],13).
```

b) `cuadrados_menores(+N, ?L)` $\leftarrow$ N es un entero positivo, L es una lista de todos los enteros positivos que tengan un cuadrado menor o igual a N . Ejemplo:

```
?- cuadrados_menores(60,[1,2,3,4,5,6,7]).
```

c) `descomposicion_cuadrada(+N, ?L)` $\leftarrow$ N es un entero positivo, L es una descomposición cuadrada de N como la definida anteriormente. Ejemplos:

```
?- descomposicion_cuadrada(30,L).
```

```
L = [1, 2, 3, 4] ;
```

```
L = [1, 2, 5]
```

```
?- descomposicion_cuadrada(31,L).
```

```
false.
```

d) `descomposicion_minima(+N, ?L)` $\leftarrow$ N es un entero positivo, L es la descomposición cuadrada de N cuya *suma de valores* (sin elevar al cuadrado) es la más baja posible. Ejemplo:

```
?- descomposicion_minima(30,?L)
```

```
L = [1, 2, 5]
```

Solución**a)**

```
suma_cuadrados(L,N):-
    suma_cuadrados(L,0,N).
```

```
suma_cuadrados([],Ac,Ac).
```

```
suma_cuadrados([I|R],Ac,S):-
    Ac2 is Ac + I*I,
    suma_cuadrados(R,Ac2,S).
```

b)

```

cuadrados_menores(N,C):-
    cuadrados_menores(N,1,C).

cuadrados_menores(N,I,[]):-
    I*I > N,!.
cuadrados_menores(N,I,[I|C]):-
    % I*I <= N,
    I1 is I + 1,
    cuadrados_menores(N,I1,C).

% alternativa
% cuadrados_menores(N,C):-
%     S is floor(sqrt(N)),findall(X,between(1,S,X),C)

```

c)

```

descomposicion_cuadrada(N,L):-
    cuadrados_menores(N,C),
    subsecuencia(C,L),
    suma_cuadrados(L,N).

subsecuencia([],[]).
subsecuencia([X|L],[X|R]):-
    subsecuencia(L,R).
subsecuencia([_|L],R):-
    subsecuencia(L,R).

```

d)

```

descomposicion_minima(N,L):-
    findall((X,C),(descomposicion_cuadrada(N,C),suma(C,X)),Decomp),
    encontrar_minima(Decomp,L).

suma(L,N):-
    suma(L,0,N).
suma([],Ac,Ac).
suma([I|R],Ac,N):-
    Ac2 is Ac + I,
    suma(R,Ac2,N).

encontrar_minima([(X,C)|Decomp],L):-
    encontrar_minima(Decomp,X,C,L).

encontrar_minima([],_,AccL,AccL).
encontrar_minima([(X,C)|Decomp],AccSum,_,L):-
    X < AccSum,!,
    encontrar_minima(Decomp,X,C,L).
encontrar_minima([(_,_)|Decomp],AccSum,AccL,L):-
    % X >= AccSum,
    encontrar_minima(Decomp,AccSum,AccL,L).

```

Ejercicio 2 [15 puntos]**a)** Indique si son verdaderas o falsas las siguientes afirmaciones. Fundamente.

i) Si dos expresiones son unificables, existe un único m.g.u. entre ellas.

FALSO: El m.g.u. no es único. Por ejemplo, $\{p(x), p(y)\}$ tiene dos posibles mgu: $\{x/y\}$ y $\{y/x\}$.

ii) Una cláusula de Horn es un tipo particular de cláusula definida.

FALSO: Las cláusulas definidas son tipos particulares de cláusulas de Horn, por definición: Una cláusula de Horn es una cláusula definida o un objetivo definido.

iii) Dados un programa lógico y un objetivo, diferentes reglas de computación pueden generar diferentes árboles SLD.

VERDADERO: Una regla de computación determina el orden en el que se van seleccionando los átomos para unificar, y esto determina las ramas del árbol que se generan.

iv) Una sustitución θ unifica un conjunto de expresiones W sii $(W\theta)$ es un conjunto no vacío.

FALSO: El conjunto $W\theta$ debe tener exactamente un elemento. El conjunto $(W\theta)$ que se obtiene de aplicar una sustitución θ a un conjunto de expresiones W puede ser no vacío sin que θ sea unificador de las expresiones.

b) Completar la siguiente definición de resolvente binaria para cláusulas de primer orden:

Sean $C1$ y $C2$ dos cláusulas de primer orden, sin variables en común:

$C1 : \{L1, \dots, Ln\}$

$C2 : \{M1, \dots, Mk\}$

y θ un mgu de Li y $\neg Mj$

La resolvente binaria es la siguiente cláusula

$C3: \{L1, \dots, Li-1, Li+1, \dots, Ln, M1, \dots, Mj-1, Mj+1, \dots, Mk\}\theta$

Ejercicio 3 [25 puntos]

Considere el siguiente programa lógico:

$f(X, Y) :- g(X, Y) .$

$f(0, 0) .$

$g(X, Y) :- p(X), q(Y) .$

$g(X, Y) :- f(X, Y) .$

$p(1) .$

$p(2) .$

$q(1) .$

$q(2) .$

$q(3) .$

i) Escriba el árbol SLD resultante de hacer la consulta $\leftarrow f(X, Y)$

ii) ¿Qué respuestas devuelve el intérprete prolog y en qué orden?

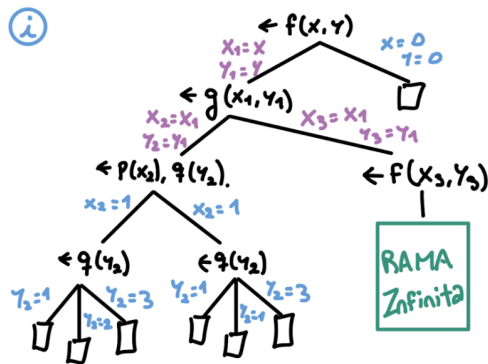
iii) Muestre cómo cambia el árbol si cambiamos la regla $g(X, Y) :- p(X), q(Y)$ por las siguientes variantes:

a) $g(X, Y) :- !, p(X), q(Y) .$

b) $g(X, Y) :- p(X), !, q(Y) .$

c) $g(X, Y) :- p(X), q(Y), ! .$

Solución



ii) Salida 1: $x=1, y=1$ (X)

Salida 2: $x=1, y=2$

Salida 3: $x=1, y=3$

Salida 4: $x=2, y=1$

Salida 5: $x=2, y=2$

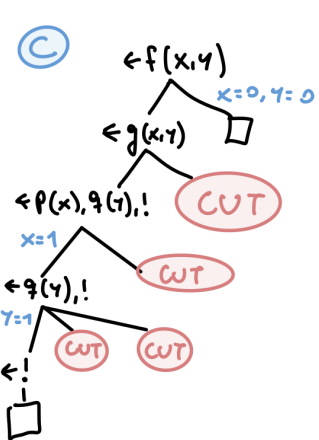
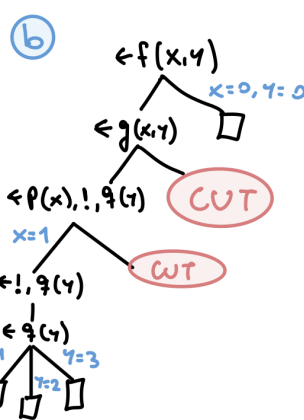
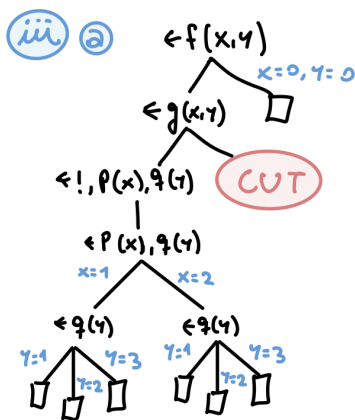
Salida 6: $x=2, y=3$

Salida 7: $x=1, y=1$ (X)

Salida 8: $x=1, y=2$

⋮

Se repiten en ciclo las salidas
Nunca se llega a la salida $x=0, y=0$



Ejercicio 4 [15 puntos]

a) Escriba una gramática lógica que reconozca expresiones aritméticas simples. Las expresiones posibles son:

- número: dígito del 0 al 9 o número negativo.
- operador binario aplicado a dos números.

Los operadores binarios son: *mas*, *menos*, *por*, *dividido*. El operador de negación es *neg*.

Ejemplos de expresiones válidas:

[5]

[neg, 3]

[3, por, 2]

[1, mas, neg, 3]

b) Escriba una gramática lógica que reconozca las expresiones de la parte a) y las evalúe. Asuma definidos los siguientes predicados:

suma(N1,N2,N3) :- N3 is N1 + N2.

diferencia(N1,N2,N3) :- N3 is N1 - N2.

producto(N1,N2,N3) :- N3 is N1 * N2.

cociente(N1,N2,N3) :- N3 is N1 / N2.

negativo(N1,N2) :- N2 is -N1.

Sugerencia: utilizar los predicados =.. y call, que permiten crear e invocar predicados.

Ejemplos de consultas:

```
?- s(Res,[1, mas, 2], []).
Res = 3.
?- s(-4, [2, por, neg, 2], []).
true.
```

Solución**a)**

```
exp_arit --> numero.
exp_arit --> numero, oper(2), numero.
numero --> num.
numero --> oper(1), num.
num --> [0]. % se puede usar between para generar los números
num --> [1].
num --> [2].
num --> [3].
num --> [4].
...
oper(2) --> [mas].
oper(2) --> [menos].
oper(2) --> [por].
oper(2) --> [dividido].
oper(1) --> [neg].
```

b)

```
exp_arit(N) --> numero(N).
exp_arit(N) --> numero(N1), oper(2, Op), numero(N2), {operar(Op, N1, N2, N)}.
numero(N) --> num(N).
numero(N) --> oper(1), num(N1), {negativo(N1, N)}.
num(0) --> [0]. % se puede usar between para generar los números
num(1) --> [1].
num(2) --> [2].
num(3) --> [3].
num(4) --> [4].
...
oper(2, suma) --> [mas].
oper(2, diferencia) --> [menos].
oper(2, producto) --> [por].
oper(2, cociente) --> [dividido].
oper(1) --> [neg].

operar(Op, N1, N2, Res) :-
    Operacion =.. [Op, N1, N2, Res],
    call(Operacion).
```